



> 1ST REVIEW · GROUP 1\_

# FraudShield

Unified Fraud Detection & Risk Scoring Platform

*Real-time ML risk scoring · Explainable alerts · Unified case management*



**Risk Score 0–100**

**D. Rohini Tanmayi**  
2520030349

**S. Manvi Vaishnavi**  
2520030326

**Varshith**  
2520030598

Under the guidance of **Dr. Y. Subbarayudu**, Assistant Professor, Dept. of CSE · Koneru Lakshmaiah Education Foundation

# Abstract

// 02

- Digital payments and e-commerce have grown rapidly — and so have the volume and sophistication of financial fraud.
- FraudShield is a unified fraud detection & risk scoring platform combining machine learning with real-time stream processing.
- A Kafka streaming pipeline ingests transactions; engineered behavioural features feed an XGBoost classifier tuned for imbalanced data.
- FastAPI, Redis and PostgreSQL deliver low-latency inference and durable case records.
- SHAP explainability and the Fraud Risk Command Center dashboard give analysts a real-time view of flagged transactions and cases.

ML

XGBoost

Kafka

SHAP

FastAPI

Real-time

0–100

risk score per transaction

7

analyst views in one dashboard

9

behavioural & context features

Banks, payment gateways and e-commerce platforms process millions of transactions every day. Machine learning combined with real-time stream processing has become the industry standard for catching fraud — this project builds that pipeline end to end, from data ingestion to an analyst-facing dashboard.

## Fraud tactics we target



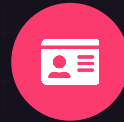
### Account takeover

New device + new  
payee + unusual  
location



### Card-not-present

High-value Web / API  
purchases



### Synthetic identity

Very young accounts  
making large transfers



### Velocity abuse

Bursts of transactions  
within 5 minutes

# Existing System vs FraudShield

// 04



## Existing (rule-based)

Static rule thresholds (amount > fixed limit)

Rules manually written; slow to adapt

High false positives block genuine users

Disconnected tools for scoring, alerts, cases

No explanation of why a rule fired



## Proposed (FraudShield)

ML risk score (0–100) learned from behaviour

Model retrained as new fraud patterns emerge

Imbalance-aware model tuned for precision & recall

One platform: monitor → alert → investigate → close

SHAP “Why was this flagged?” for every transaction

| Approach                    | Strength                               | Limitation                   | FraudShield takes         |
|-----------------------------|----------------------------------------|------------------------------|---------------------------|
| Rule-based expert systems   | Simple, auditable                      | Static; high false positives | Rules only as features    |
| Logistic Regression / SVM   | Fast, interpretable                    | Misses non-linear patterns   | Baseline comparison       |
| Random Forest               | Robust, handles mixed data             | Weaker on rare classes       | Prototype dashboard model |
| Gradient Boosting (XGBoost) | State of the art on tabular fraud data | Needs tuning for imbalance   | Primary classifier        |
| Deep / Autoencoder models   | Learns anomalies                       | Black-box, heavy compute     | Future work               |
| SHAP explainability         | Per-prediction reasons                 | Extra compute per score      | “Why flagged” view        |

# Problem Statement



## Real-time accuracy

How can suspicious transactions be identified accurately and in real time, without rigid static rules?



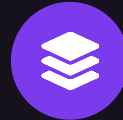
## Fewer false positives

How can false positives be reduced while still catching sophisticated, evolving fraud patterns?



## Explainability

How can analysts be given clear, explainable reasons for every flagged transaction?



## Unification

How can monitoring, alerting and case management live in a single platform?

# Solution — System Architecture

// 07



## Kafka

Ingests transaction events in real time



## Feature Engine

Velocity, distance, device, payee, account age



## XGBoost + SHAP

Risk score 0–100 with feature contributions



## FastAPI

Low-latency scoring & case APIs



## Redis · PostgreSQL

Hot cache + durable case records



## Command Center

Dashboard, alerts & case triage

**Risk bands drive action:** **Low** → approve automatically · **Medium** → under review alert · **High** → flagged, case opened for an analyst

*Every decision is stored with its SHAP explanation so analysts can audit why it was made.*

# Objectives

1

**Build** a feature-engineered, imbalance-aware ML model that reliably flags fraudulent transactions.

2

**Design** a low-latency pipeline (Kafka → FastAPI → Redis / PostgreSQL) for real-time inference.

3

**Provide** a Fraud Risk Command Center for live monitoring, alerting and case triage.

4

**Explain** every model decision to non-technical fraud analysts using SHAP.

5

**Evaluate** the system on precision, recall, F1-score and AUC-ROC.

## Evaluation metrics

### Precision

of flagged, how many are fraud

### Recall

of fraud, how much we catch

### F1-score

balance of the two

### AUC-ROC

ranking quality across thresholds



## Data Pipeline & ML Model

*D. Rohini Tanmayi*

- Feature engineering
- Class-imbalance handling (imbalanced-learn)
- XGBoost training & tuning
- SHAP explanations per transaction



## Streaming, Backend API & Data Layer

*S. Manvi Vaishnavi*

- Kafka transaction ingestion
- FastAPI scoring & case endpoints
- Redis caching of hot features
- PostgreSQL schema for transactions & cases



## Dashboard & Explainability UI

*Varshith*

- Fraud Risk Command Center
- Live monitoring & transaction detail
- Alerts and case management views
- Score-a-transaction & model views

# Command Center — Dashboard Views

// 10



## Dashboard

KPIs, risk distribution, fraud trend, top risk factors, live feed, alerts by channel



## Live Monitoring

Per-transaction risk gauge, score breakdown and SHAP chart



## Transactions

ID, user, amount, channel, location, risk score, status, time



## Alerts

High / medium / low counts and flagged & under-review queue



## Cases

Priority, status, assignee; filter Open → Closed



## Score Transaction

Enter a transaction, get an instant score + explanation



## Models

Performance metrics, feature importances, demo reset

# Risk Scoring Inputs & Features

Each transaction is scored from behavioural and context signals — the same inputs captured on the Score Transaction screen.

## Amount

Transaction value (\$)

## Distance

km from home location

## Velocity

Txns in the last 5 min

## Account age

Days since account opened

## Hour of day

0–23; odd-hour activity

## New device

First seen on this device

## New payee

First payment to recipient

## Channel

Mobile · Web · API · POS

## Location

City / country of txn

### Example: high-risk transaction

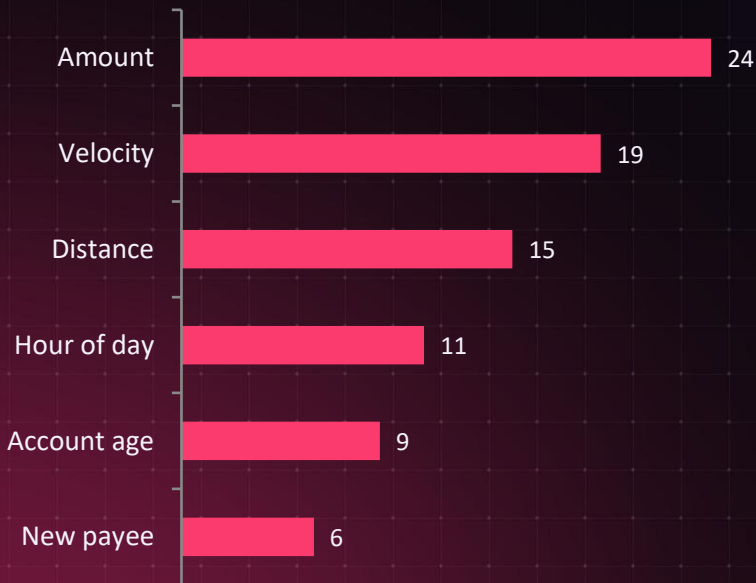
- \$8,950 at midnight
- 1,200 km from home
- 5 txns in 5 minutes
- 20-day-old account
- New device + new payee

### → High risk

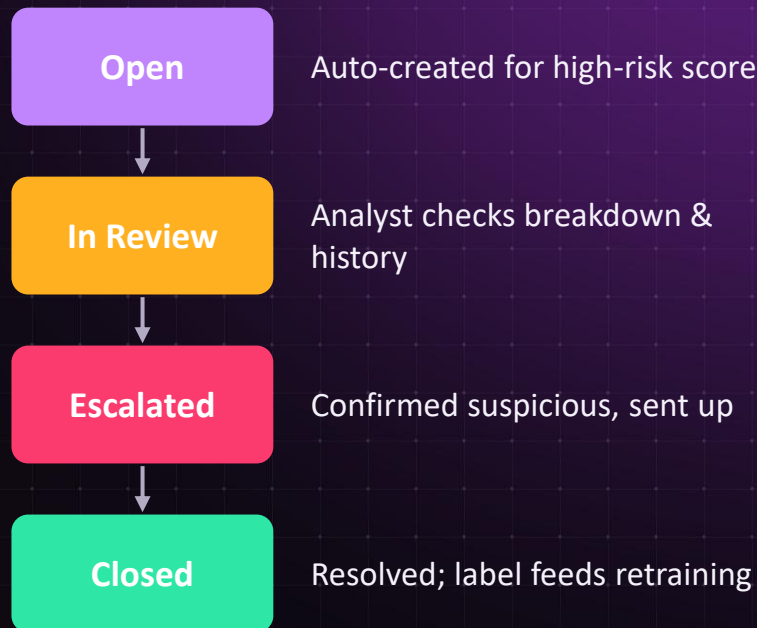
Flagged and sent to case queue

## “Why was this flagged?”

*Illustrative SHAP contributions for one flagged transaction*



## Case lifecycle



# Transaction Data Flow — Step by Step

// 13

1

## Customer pays

Mobile / Web / API / POS transaction is created

2

## Event streamed

Kafka producer publishes it to the transactions topic

3

## Features built

Velocity, distance, account age, device & payee checks (Redis lookups)

4

## Model scores

XGBoost returns fraud probability → scaled to 0–100

5

## Explain

SHAP computes the contribution of each feature

6

## Decide & store

Band assigned, record saved to PostgreSQL, alert raised if needed

7

## Analyst acts

Dashboard shows alert; analyst opens case and resolves it

Target latency: < 100 ms per transaction

# Database Design (PostgreSQL)

// 14

## users

```
user_id PK
name
home_location
created_at
risk_profile
```

## transactions

```
txn_id PK
user_id FK
amount
channel
location
device_id
risk_score
status
created_at
```

## cases

```
case_id PK
txn_id FK
priority
status
assigned_to
resolved_at
```

## explanations

```
txn_id FK
feature
value
shap_value
```

**Redis cache:** per-user velocity counters (5-min window), last device, known payees — so features are computed in milliseconds without hitting PostgreSQL.

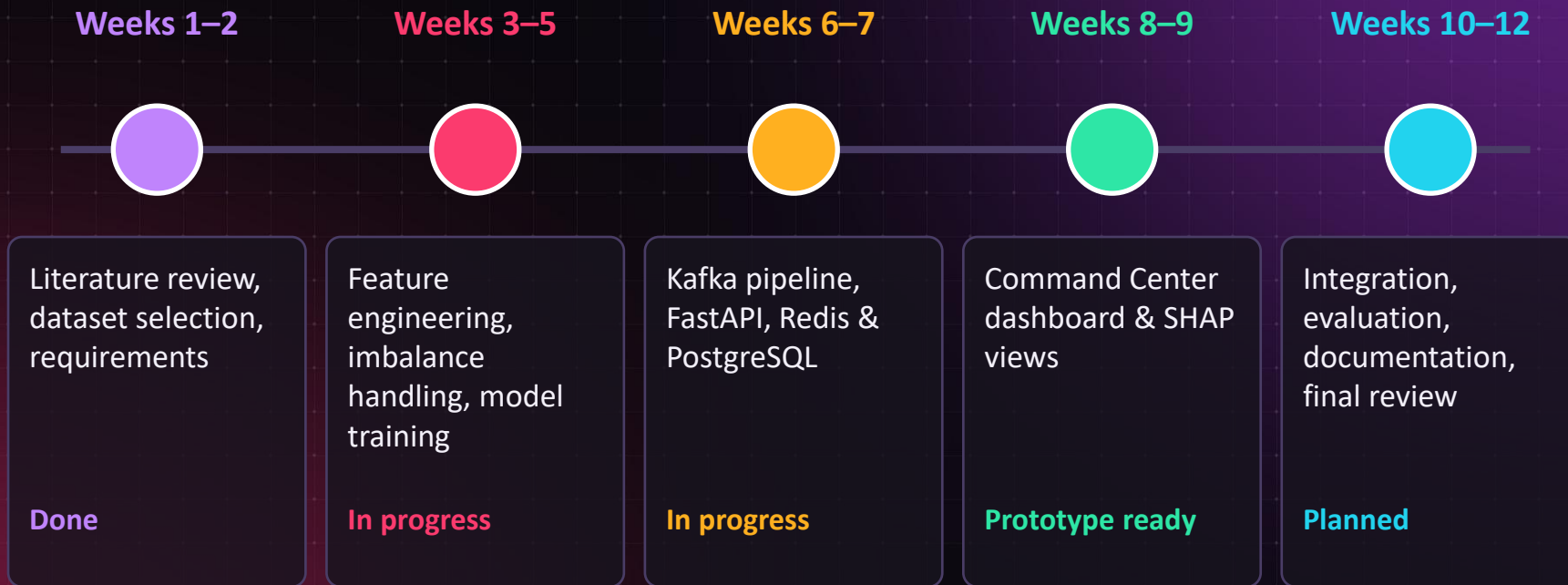
# Backend API Endpoints (FastAPI)

// 15

| Method | Endpoint           | Purpose                                              |
|--------|--------------------|------------------------------------------------------|
| POST   | /score             | Score a transaction → risk score, band, SHAP reasons |
| GET    | /transactions      | List recent transactions with scores & status        |
| GET    | /transactions/{id} | Full breakdown for Live Monitoring view              |
| GET    | /alerts            | High / medium / low alert counts and queue           |
| GET    | /cases             | Cases filtered by status (Open, In Review...)        |
| PATCH  | /cases/{id}        | Update status, assignee, analyst notes               |
| GET    | /model/metrics     | Precision, recall, F1, AUC & feature importances     |
| POST   | /demo/reset        | Re-seed demo transactions and cases                  |

# Project Timeline

// 16

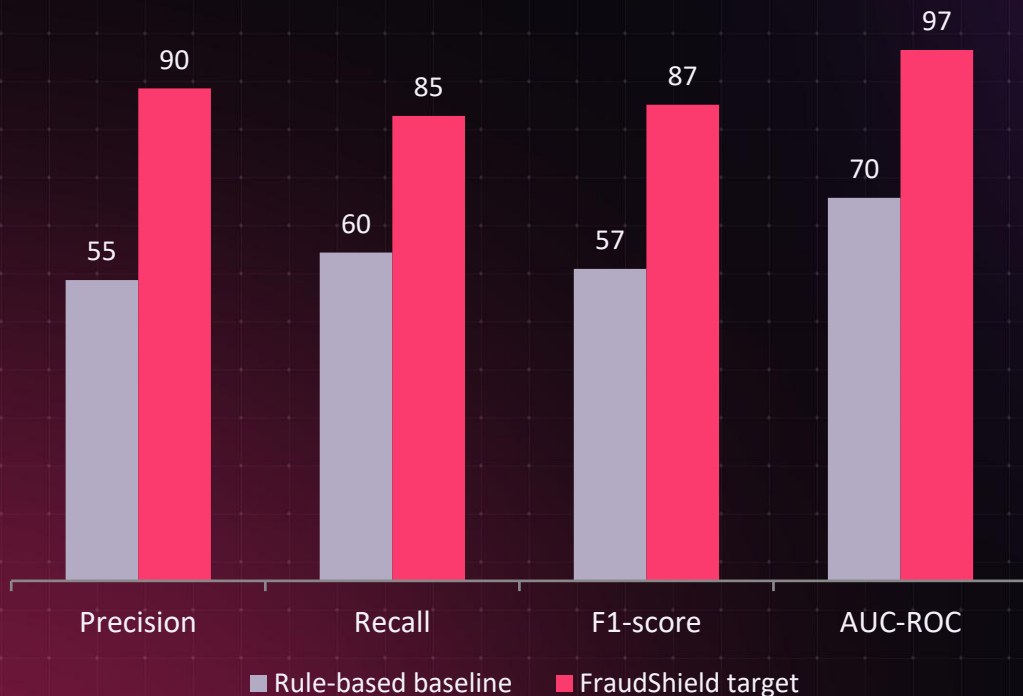


*Current status: dashboard prototype complete; ML and backend integration in progress for the 2nd review.*

# Expected Results

// 17

Target metrics (%) vs rule-based baseline



**< 100 ms**

scoring latency

↓ **40%**

false positives vs rules

**100%**

alerts with an explanation

*Targets to be validated on the evaluation dataset in the next review.*



## Languages

Python 3.10+, JavaScript (ES6+)



## ML libraries

pandas, scikit-learn, XGBoost, SHAP, imbalanced-learn



## Backend

FastAPI, Uvicorn, SQLAlchemy



## Streaming

Apache Kafka 3.x



## Database

PostgreSQL 14+, Redis 7+



## Frontend

React.js, Recharts, Axios, Chart.js



## Tools

VS Code, Git/GitHub, Postman, Docker

## Conclusion

- A complete, industry-style fraud pipeline from real-time ingestion to explainable dashboards.
- Real-time risk scoring, explainable predictions and unified case management in one platform.
- Analysts see not just a score, but exactly why each transaction was flagged.



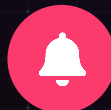
### Feedback retraining

Retrain the model on analyst-confirmed case labels



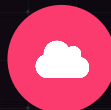
### Graph features

Detect fraud rings across linked accounts & devices



### Multi-channel alerts

Email / SMS / Slack notifications for high-risk events



### Cloud scale

Kubernetes deployment for production throughput



# Thank You

Questions & Discussion

FraudShield · Group 1 · Dept. of CSE, KL University